ChatGPT

Генетични алгоритми: йерархична структура и съдържание за статичен сайт

Резюме

Този документ дефинира навигационна йерархия (страници и подстраници) за статичен уебсайт за **генетични алгоритми (ГА)**, като за всяко навигационно ниво дава: **navigation label**, **препоръчан URL slug**, и **какво да съдържа страницата** (резюме, целева аудитория, ориентировъчна дължина, нужни активи). Структурата покрива всички поисканите теми: теория (вкл. schema theorem и теоретични модели), оператори (selection/crossover/mutation), представяния и стратегии за кодиране, фитнес функции, настройка и контрол на параметри, сходимост и разнообразие, хибридни методи, многоцелеви ГА (NSGA-II/III и др.), обработка на ограничения, реални приложения, бенчмаркове, метрики, сложност, имплементационни детайли/псевдокод, библиотеки/инструменти, казуси и чести капани. Подборът на препоръчани четива приоритизира **първични/официални източници** и **български материали**, когато са налични. [1]

Карта на сайта

Таблицата по-долу е „site map“ за статичен сайт. Колони: **ID** (йерархична номерация), **Page label** (как да се вижда в менюто), **Slug**, **Breadcrumb** (пътека), **Ниво** (Beginner/Mixed/Advanced). (Slug-овете са предложения; приемат се еквиваленти според вашите URL-конвенции.)

ID	Навигационен етикет (label)	Препоръчан slug	Breadcrumb	Ниво
1	Начало	`/`	Начало	Beginner
2	Какво е генетичен алгоритъм	`/ga/`	Начало › ГА	Beginner
2.1	Каноничен процес	`/ga/kanonichen-proces/`	Начало › ГА › Каноничен процес	Beginner
2.2	Кога ГА работи/не работи	`/ga/koga-raboti/`	Начало › ГА › Кога работи	Mixed
3	Теоретични основи	`/teoriya/`	Начало › Теория	Mixed
3.1	Формален модел и терминология	`/teoriya/formalen-model/`	Начало › Теория › Формален модел	Mixed
3.2	Схеми и Schema Theorem	`/teoriya/schema-theorem/`	Начало › Теория › Schema theorem	Advanced

ID	Навигационен етикет (label)	Препоръчан slug	Breadcrumb	Ниво
3.3	Маркови модели и сходимост	`/teoriya/sxodimost/`	Начало › Теория › Сходимост	Advanced
3.4	No Free Lunch	`/teoriya/no-free-lunch/`	Начало › Теория › NFL	Mixed
4	Представяния и кодиране	`/predstavyane/`	Начало › Представяния	Mixed
4.1	Бинарно и дискретно кодиране	`/predstavyane/binarno-diskretno/`	Начало › Представяния › Бинарно	Beginner
4.2	Реално-кодирани ГА	`/predstavyane/realno-kodirani/`	Начало › Представяния › Реално-кодирани	Mixed
4.3	Пермутационни представяния	`/predstavyane/permutacionni/`	Начало › Представяния › Пермутации	Mixed
4.4	Random keys и индиректно кодиране	`/predstavyane/random-keys/`	Начало › Представяния › Random keys	Advanced
4.5	Дървета и Genetic Programming	`/predstavyane/genetic-programming/`	Начало › Представяния › GP	Mixed
5	Оператори	`/operatori/`	Начало › Оператори	Mixed
5.1	Селекция	`/operatori/selekciya/`	Начало › Оператори › Селекция	Mixed
5.2	Кръстосване	`/operatori/krastosvane/`	Начало › Оператори › Кръстосване	Mixed
5.3	Мутация	`/operatori/mutaciya/`	Начало › Оператори › Мутация	Mixed
6	Фитнес и целеви функции	`/fitnes/`	Начало › Фитнес	Mixed
6.1	Дефиниране и скалиране	`/fitnes/skalirane/`	Начало › Фитнес › Скалиране	Mixed
6.2	Ограничения	`/fitnes/ogranicheniya/`	Начало › Фитнес › Ограничения	Advanced
7	Параметри и динамика	`/parametri/`	Начало › Параметри	Mixed

ID	Навигационен етикет (label)	Препоръчан slug	Breadcrumb	Ниво
7.1	Настройване и контрол на параметри	`/parametri/tuning-control/`	Начало › Параметри › Контрол	Advanced
7.2	Сходимост и преждевременна конвергенция	`/parametri/sxodimost-diversitet/`	Начало › Параметри › Сходимост и разнообразие	Mixed
7.3	Модели за разнообразие	`/parametri/diversitet-mehanizmi/`	Начало › Параметри › Механизми	Advanced
8	Разширени методи	`/razshireniya/`	Начало › Разширения	Mixed
8.1	Хибридни и меметични алгоритми	`/razshireniya/memetichni/`	Начало › Разширения › Меметични	Advanced
8.2	Многоцелеви ГА	`/razshireniya/multiobjective/`	Начало › Разширения › Многоцелеви	Mixed
8.2.1	NSGA-II	`/razshireniya/multiobjective/nsga-ii/`	Начало › Разширения › Многоцелеви › NSGA-II	Advanced
8.2.2	NSGA-III	`/razshireniya/multiobjective/nsga-iii/`	Начало › Разширения › Многоцелеви › NSGA-III	Advanced
8.2.3	MOEA/D, SPEA2, IBEA	`/razshireniya/multiobjective/drugi-moea/`	Начало › Разширения › Многоцелеви › Други	Advanced
9	Оценяване и бенчмаркове	`/ocenyavane/`	Начало › Оценяване	Mixed
9.1	Метрики	`/ocenyavane/metriki/`	Начало › Оценяване › Метрики	Mixed
9.1.1	Едноцелеви метрики	`/ocenyavane/metriki/ednocel/`	Начало › Оценяване › Метрики › Едноцелеви	Mixed
9.1.2	Многоцелеви метрики	`/ocenyavane/metriki/mnogocel/`	Начало › Оценяване › Метрики › Многоцелеви	Advanced
9.2	Бенчмаркове COCO/BBOB	`/ocenyavane/benchmarks/coco-bbob/`	Начало › Оценяване › Бенчмаркове › COCO/BBOB	Mixed

ID	Навигационен етикет (label)	Препоръчан slug	Breadcrumb	Ниво
9.3	ZDT, DTLZ, WFG	`/ocenyavane/benchmarks/zdt-dtlz-wfg/`	Начало › Оценяване › Бенчмаркове › ZDT/DTLZ/WFG	Advanced
9.4	Сложност и бюджет на оценки	`/ocenyavane/slojnost/`	Начало › Оценяване › Сложност	Mixed
10	Имплементация и инструменти	`/implementaciya/`	Начало › Имплементация	Mixed
10.1	Псевдокод и шаблони	`/implementaciya/psevdokod/`	Начало › Имплементация › Псевдокод	Beginner
10.2	Библиотеки и tooling	`/implementaciya/biblioteki/`	Начало › Имплементация › Библиотеки	Mixed
10.3	Повторяемост и експериментален протокол	`/implementaciya/reproducibility/`	Начало › Имплементация › Reproducibility	Advanced
11	Приложения и казуси	`/prilozheniya/`	Начало › Приложения	Mixed
11.1	Управление и инженерна оптимизация	`/prilozheniya/inzhenerstvo-upravlenie/`	Начало › Приложения › Инженерство	Mixed
11.2	Оптимизация на SQL заявки	`/prilozheniya/sql/`	Начало › Приложения › SQL	Advanced
11.3	Генетични размити системи	`/prilozheniya/genetichni-razmiti-sistemi/`	Начало › Приложения › Размити системи	Mixed
12	Чести капани и добри практики	`/kapani/`	Начало › Капани	Mixed
12.1	Типични грешки при дизайн	`/kapani/dizayn/`	Начало › Капани › Дизайн	Mixed
12.2	Типични грешки при оценяване	`/kapani/ocenyavane/`	Начало › Капани › Оценяване	Advanced
13	Източници	`/iztochnici/`	Начало › Източници	Mixed

Покритието на научните „стълбове“ е структурирано така: класическа основа (Holland, Goldberg, schema theorem), формална теория (Vose, модели на сходимост), многоцелеви методи (Deb, NSGA-II/

III), ограничения (Deb 2000), принципни граници (Wolpert & Macready NFL), и стандартизирано benchmarking/метрики (COCO/BBOB, ZDT/DTLZ/WFG, hypervolume). [2]

Навигационни диаграми

Mermaid: навигация на сайта

```
flowchart TD
  A[Начало /] --> B[ГА /ga/]
  A --> C[Теория /teoriya/]
  A --> D[Представяния /predstavyane/]
  A --> E[Оператори /operatori/]
  A --> F[Фитнес /fitnes/]
  A --> G[Параметри /parametri/]
  A --> H[Разширения /razshireniya/]
  A --> I[Оценяване /ocenyavane/]
  A --> J[Имплементация /implementaciya/]
  A --> K[Приложения /prilozheniya/]
  A --> L[Капани /kapani/]
  A --> M[Източници /iztochnici/]

  C --> C1[Формален модел]
  C --> C2[Schema theorem]
  C --> C3[Сходимост]
  C --> C4[NFL]

  H --> H1[Меметични]
  H --> H2[Многоцелеви]
  H2 --> H21[NSGA-II]
  H2 --> H22[NSGA-III]
  H2 --> H23[Други MOEA]
```

Диаграмата е проектирана за „главно меню“ + „подменю“ в секции с висок когнитивен товар (теория и многоцелеви методи). [3]

Mermaid: каноничен GA процес

```
flowchart TD
  A[Инициализация на популация P0] --> B[Оценка: f(x) / fitness]
  B --> C{Стоп критерий?}
  C -- Да --> Z[Резултат: най-добро решение / Pareto множество]
  C -- Не --> D[Селекция на родители]
  D --> E[Кръстосване (crossover)]
  E --> F[Мутация (mutation)]
  F --> G[Оценка на потомството]
```

```
G --> H[Замяна/оцеляване + елитизъм]
H --> B
```

Това е каноничният цикъл, описван в класическите уводи и туториали за ГА и служи като „референтен модел“ за всички разширения (многоцелеви, constrained, island models, хибриди). [4]

Спецификация на съдържанието по страници и подстраници

По-долу всяка позиция е представена като „page spec“ и включва (1) метаданни за навигация и производство на съдържание и (2) минимален „ядрен комплект“: обяснение на разбираем език, ключови уравнения (където е релевантно), кратък пример, препоръчано четиво (първично/официално + българско при възможност). Избраните формули са **концептуално-ключови** и са предназначени за възпроизводимост и контролируемост на имплементациите. [5]

1 Начало

Navigation label: Начало
Slug: `/`
Какво съдържа: кратко позициониране на сайта, “как да четем/ползваме”, карта на сайта, 2–3 типични сценария (инженерна оптимизация, комбинаторика, многоцелеви дизайн).
Аудитория: Beginner
Дължина: ~700–1200 думи
Активи (само списък, без вграждане): - `site-navigation.svg` — „Йерархия на страниците (карта на сайта)“ - `ga-process-flow.svg` — „Каноничен процес на генетичен алгоритъм (flowchart)“

Обяснение (plain-language):
Генетичният алгоритъм е метод за оптимизация, който поддържа **популация** от кандидат-решения и итеративно я подобрява чрез селекция (производство на родители), вариация (кръстосване и мутация) и замяна (кои индивиди остават). Той е част от по-широката област на еволюционните алгоритми и се използва, когато целевата функция е сложна, дискретна, шумна или „черна кутия“. [6]

Ключови уравнения:
- Обща постановка (максимизация): $\max_{x \in X} f(x)$.
- При работа с популация P_t: измерваме $f(x)$ за всеки индивид и използваме това за селекция. [7]

Пример (кратък):
OneMax (учебен пример): $f(x) = \sum_{i=1}^{L} x_i$ за бинарен низ $x \in \{0,1\}^L$. ГА обикновено „изкачва“ към низ от всички единици чрез избор на по-добри индивиди и поддържане на разнообразие с мутация. [8]

Препоръчано четиво:
- Увод и общ модел: Whitley tutorial. [9]
- Класическа книга за ГА практика/концепции: Goldberg (1989) (ACM record). [10]
- Български увод/дипломна работа с имплементационни идеи: Попов (2003). [11]

---

2 Какво е генетичен алгоритъм

Navigation label: ГА
Slug: `/ga/`
Какво съдържа: дефиниция, ключови понятия (ген/хромозома/генотип/фенотип), типове задачи, кратка история и място в „evolutionary computation“.
Аудитория: Beginner
Дължина: ~1500–2500 думи
Активи: - `ga-terminology-diagram.svg` — „Генотип ↔ фенотип и декодиране“ - `fitness-landscape-intro.png` — „Интуитивен фитнес ландшафт (локални/глобални оптимуми)“

Обяснение:
ГА моделира „адаптация“ чрез селекция и вариация върху популация от решения. Класическият възглед и мотивация са систематизирани в ранните фундаментални разработки за адаптация в изкуствени системи и в класическите инженерни изложения на ГА като метод за търсене и оптимизация. [12]

Ключови уравнения:
- Декодиране: $\phi : G \to X$, където G е пространство на кодирани решения (генотипи), X е пространство на реални решения (фенотипи).
- Емпирична оценка: $F(g) = f(\phi(g))$. [13]

Пример:
Оптимизация на параметри на регулатор (контрол) чрез ГА: популацията са кандидати за параметри, fitness е функционал за грешка/енергия; мутацията предотвратява „заключване“ в локално решение. Български пример на приложна дипломна работа: „Генетични алгоритми за оптимизация – приложение в задачата за синтез на регулатор“. [14]

Препоръчано четиво:
- Holland (MIT Press) като исторически фундамент. [15]
- Попов (2003) като български ресурс. [16]

2.1 Каноничен процес

Navigation label: Каноничен процес
Slug: `/ga/kanonichen-proces/`
Какво съдържа: разширено обяснение на цикъла (инициализация, оценка, селекция, кръстосване, мутация, замяна, стоп), типове GA (generational vs steady-state), базови избори.
Аудитория: Beginner
Дължина: ~1500–2500 думи
Активи: - `ga-process-flow.svg` — „Flowchart на каноничен GA“ - `ga-generational-vs-steadystate.svg` — „Generational vs steady-state replacement“

Обяснение:
„Каноничен“ GA е референтен модел: повечето реални варианти са модификации върху него (друга селекция, други оператори, елитизъм, островни модели, multiobjective survival). Whitley систематизира този модел и показва и варианти като island/cellular GA. [9]

Ключови уравнения:
- Бюджетът често се измерва в **брой fitness оценки**: $B \approx N \times G$ (за generational GA с N индивида и G поколения), което е критично при „скъпа“ оценка. [17]

Пример:
С популация $N = 100$ и $G = 500$ имате ~50 000 оценки на fitness (ако оценявате всяко потомство). Това е директна връзка между параметри и цена на експеримента. [18]

Препоръчано четиво:
- Whitley tutorial (каноничен GA + island модели). [9]
- Goldberg (1989) (книга за оператори, fitness scaling и практическо приложение). [10]

2.2 Кога ГА работи/не работи

Navigation label: Кога работи
Slug: `/ga/koga-raboti/`
Какво съдържа: критерии за пригодност: дискретни/комбинаторни пространства, недиференцируеми цели, шум; кога да предпочетете други методи; връзка с No Free Lunch.
Аудитория: Mixed
Дължина: ~1500–2500 думи
Активи: - `fitness-landscape-modalities.png` — „Мултимодален ландшафт и локални оптимуми“
- `nfl-intuition-diagram.svg` — „Интуиция за No Free Lunch“

Обяснение:
ГА е полезен, когато имате широки пространства и операционално можете да дефинирате валидни оператори за вариация. Важно е да се комуникира принципно ограничение: при усредняване върху всички възможни задачи няма универсално „най-добър“ оптимизатор (No Free Lunch). Затова ГА печели чрез **индуктивен байъс** (кодиране, оператори, домейн знания) към вашия клас проблеми. [19]

Ключови уравнения:
NFL се формулира чрез равенства на усреднената производителност върху множество функции; страницата следва да дава определения за “set of objective functions” и “performance measure” и да показва изводите на Wolpert & Macready. [20]

Пример:
Ако сравнявате GA срещу специфичен локален метод на клас функции „гладки, конвексни“, локалният метод обикновено има фундаментално предимство. Ако задачите са пермутационни/NP-трудни (напр. join ordering в SQL), GA може да е конкурентен със специализирано кодиране/оператори. [21]

Препоръчано четиво:
- Wolpert & Macready (NFL). [22]
- Български казус за SQL/комбинаторика: Велев (СУ). [23]

---

3 Теоретични основи

Navigation label: Теория
Slug: `/teoriya/`
Какво съдържа: вход към теорията: (1) формален модел и термини, (2) schema theorem и ограниченията му, (3) Маркови/точни модели и сходимост, (4) No Free Lunch и дизайн-следствия.
Аудитория: Mixed
Дължина: ~1200–2000 думи (като hub към подстраници)
Активи: - `theory-map.svg` — „Карта: schema theorem ↔ маркови модели ↔ NFL“

Обяснение:
Теоретичният раздел трябва да разделя ясно: (i) **класическа интуиция** (schemas/building blocks) и (ii) **формални резултати** (точни модели, стационарни разпределения, условия за сходимост при елитизъм). В практиката тези линии се комбинират за правилни инженерни избори (размер на популация, оператори, параметри). [24]

3.1 Формален модел и терминология

Navigation label: Формален модел
Slug: `/teoriya/formalen-model/`
Какво съдържа: формална постановка (генотип/фенотип, декодиране, fitness), разлики GA/EA, generational vs steady-state, селекция и замяна като оператори.
Аудитория: Mixed
Дължина: ~2000–3500 думи
Активи: - `genotype-phenotype-map.svg` — „Генотип–фенотип и декодиране“ - `selection-replacement-operators.svg` — „Селекция vs оцеляване (survivor selection)“

Обяснение:
Добра „формална страница“ дефинира пространствата G (генотипи) и X (решения) и декодиране ϕ. Отделя „mating selection“ (кой става родител) от “survivor selection” (кой оцелява), защото различни алгоритми (вкл. NSGA-II) променят именно survival слоя. [25]

Ключови уравнения:
- $\max_{x \in X} f(x)$, $F(g) = f(\phi(g))$.
- При стохастични fitness оценки: $F(g) = \mathbb{E}[f(\phi(g), \omega)]$ и практическо приближение чрез средно от симулации. [13]

Пример:
За пермутационен проблем ϕ може да е “repair/decoder”, който превръща произволен вектор в валидна пермутация (напр. random keys чрез сортиране). [26]

Препоръчано четиво:
- Vose (SGA като математически обект). [27]
- Whitley tutorial (термини и модели). [9]

3.2 Схеми и Schema Theorem

Navigation label: Schema theorem
Slug: `/teoriya/schema-theorem/`
Какво съдържа: дефиниция на schema, ред и дефинираща дължина, schema theorem (като неравенство), интуиция за building blocks, критики/граници на приложимост.
Аудитория: Advanced
Дължина: ~2500–4500 думи
Активи: - `schema-example.svg` — „Schema над {0,1,*} и ред/дължина“ - `building-blocks-intuition.svg` — „Комбиниране на подструктури чрез crossover“

Обяснение:
Schema theorem е исторически централен аргумент за това защо GA е ефективен: селекцията увеличава „добри“ schemas, докато crossover и mutation ги разрушават с вероятност, зависеща от дефиниращата дължина и реда. Важното в модерна експозиция е да се подчертае: това е **еднопоколенчески очакван долен праг** (не точна динамика) и затова се комбинира с по-точни модели (Markov/SGA теория). [28]

Ключови уравнения (какво да включи страницата):
- Schema $H \in \{0, 1, *\}^L$, ред $o(H)$, дефиниираща дължина $\delta(H)$.
- Канонична форма на неравенство за очаквания брой копия $m(H, t)$ в следващо поколение (включва селекция + disruption от crossover/mutation). Препоръчано е да се даде derivation по Whitley + връзка с Holland. [29]

Пример:
Ако $H = 1 * *0 * * * * * 1$, то $o(H) = 3$. При bit-flip мутация с вероятност p_m, вероятността schema да оцелее от мутация приблизително е $(1 - p_m)^{o(H)}$. Така високите p_m рушат експлоатацията. [7]

Препоръчано четиво:
- Holland (исторически фундамент). [15]
- Whitley tutorial (ясна derivation и критичен контекст). [9]
- Vose (точни модели на SGA). [27]

3.3 Маркови модели и сходимост

Navigation label: Сходимост
Slug: `/teoriya/sxodimost/`
Какво съдържа: моделиране на GA като Маркова верига върху популации/разпределения; какво значи „сходимост“ при наличие на мутация; условия за конвергенция при елитизъм; връзка теория ↔ практическо спиране.
Аудитория: Advanced
Дължина: ~3000–5000 думи
Активи: - `markov-chain-ga.svg` — „GA като Марков процес върху популации“ - `convergence-vs-stationary.svg` — „Конвергенция към оптимум vs стационарно разпределение“ - `elitism-convergence-diagram.svg` — „Роля на елитизма“

Обяснение:
В строг смисъл, GA с ненулева мутация обикновено не е „абсорбиращ“ в оптимума (има ненулев шанс да напусне оптималното решение). Въпреки това, при определени условия (особено **елитистка** селекция/оцеляване) може да се докаже конвергенция към глобалния оптимум (в подходящ смисъл) в общи пространства. Това дисциплинира твърденията за „гаранции“ и насочва вниманието към вероятност за успех до бюджет. [30]

Ключови уравнения:
- Състояние S_t (популация) и преход $P(S_{t+1} \mid S_t)$.
- Практическа метрика: $P(\text{успех до } B)$ и очаквано време до достигане на цел (run-time). Свържете с ERT в benchmarking. [31]

Пример:
Ако включите елитизъм (запазване на най-добрия индивид), шансът да „загубите“ най-доброто решение в следващи поколения се намалява, което е важно при шум. Теорията на Rudolph дава условия за конвергенция при елитистко правило. [32]

Препоръчано четиво:
- Vose (SGA theory). [27]
- Rudolph (конвергенция при елитистки EA). [33]
- Whitley (контекст и практични implications). [9]

3.4 No Free Lunch

Navigation label: NFL
Slug: `/teoriya/no-free-lunch/`
Какво съдържа: формулировка; как се интерпретира правилно; защо това не обезсмисля GA, а дефинира нуждата от bias/структура; връзка с кодиране и оператори.
Аудитория: Mixed
Дължина: ~1800–3000 думи
Активи: - `nfl-diagram.svg` — „Компенсация на производителност между класове задачи“ - `bias-vs-generalization.svg` — „Индуктивен байъс: кодиране/оператори“

Обяснение:
No Free Lunch теоремите показват, че без предположения за класа проблеми няма универсално най-добър алгоритъм: успехът на GA идва от това, че дизайнът му (представяне, оператори, fitness shaping) вкарва структурирано предположение за задачата. Това е и методологичен аргумент за прозрачна експериментална методика: сравнявате алгоритми върху ясно дефиниран клас бенчмаркове. [34]

Ключови уравнения:
Страницата трябва да включи дефиниции за множество функции F, алгоритми A, и performance measure, както е в Wolpert & Macready. [22]

Пример:
GA, конфигуриран за пермутационни задачи (TSP-style оператори), може да е силен там, но може да е слаб за гладка конвексна оптимизация, където методи с градиент/квазинютон доминират. [35]

Препоръчано четиво:
- Wolpert & Macready (официален PDF). [22]
- COCO/BBOB като пример за стандартизирано сравнение на алгоритми. [36]

---

4 Представяния и кодиране

Navigation label: Представяния
Slug: `/predstavyane/`
Какво съдържа: принципът „representation drives search“ (кодиране определя геометрията на търсенето); типове представяния; индиректни кодирания (random keys); GP.
Аудитория: Mixed
Дължина: ~1500–2500 думи (hub към подстраници)
Активи: - `representation-overview-table.png` — „Таблица: представяния и оператори“ - `genotype-to-phenotype-examples.svg` — „Примери за декодиране“

Обяснение:
Представянето е „контракт“ между задачата и операторите: неправилно кодиране може да направи crossover разрушителен и да генерира невалидни решения (особено при пермутации). Индиректните кодирания (random keys) са класически начин да се гарантира валидност чрез декодер. [37]

4.1 Бинарно и дискретно кодиране

Navigation label: Бинарно/дискретно
Slug: `/predstavyane/binarno-diskretno/`
Какво съдържа: бинарни низове, категорийни/целочислени гени, Hamming distance, схеми; кога е удобно и кога създава „клифове“.
Аудитория: Beginner
Дължина: ~2000–3500 думи
Активи: - `binary-encoding-schema.svg` — „Винарен низ и schema“ - `hamming-distance-illustration.svg` — „Hamming distance и малки промени“

Обяснение:
Бинарното кодиране е исторически стандарт и е най-съвместимо със schema-based теория. Практически е удобно за selection на характеристики (feature selection), логически дизайни и дискретни решения. [38]

Ключови уравнения:
- OneMax: $f(x) = \sum_{i=1}^{L} x_i$. [39]
- Bit-flip mutation: за всеки бит i: $x_i \leftarrow 1 - x_i$ с вероятност p_m. [9]

Пример:
„Избор на подмножество от признаци“: ген $x_i = 1$ означава включен признак i, fitness измерва точност/компромис точност–сложност. [9]

Препоръчано четиво:
- Mitchell (MIT Press) за базови представяния и примери. [40]
- Попов (2003) за български контекст и примери. [16]

4.2 Реално-кодирани ГА

Navigation label: Реално-кодирани
Slug: `/predstavyane/realno-kodirani/`
Какво съдържа: реални вектори, box constraints, SBX, полиномиална/гаусова мутация, distribution indices; избягване на дискретизация.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `sbx-illustration.svg` — „SBX: генериране на деца около родители“ - `polynomial-mutation-illustration.svg` — „Polynomial mutation (интуитивна)“ - `box-constraints-clipping.svg` — „Box constraints: clip/reflect/repair“

Обяснение:
Реално-кодирани ГА работят директно върху реални параметри и са популярни в инженерни оптимизации. SBX (Simulated Binary Crossover) е класически оператор за реални променливи, разработен да имитира свойствата на бинарния single-point crossover. [41]

Ключови уравнения (SBX – какво да съдържа страницата):
Страницата трябва да въведе distribution index η_c и преобразуването на случаен $u \in (0,1)$ към spread factor β_q, след което да даде формулите за децата:

$$y_1 = 0.5[(1+\beta_q)x_1 + (1-\beta_q)x_2], \quad y_2 = 0.5[(1-\beta_q)x_1 + (1+\beta_q)x_2].$$

Това е стандартната форма в описанията на SBX и се разглежда в оригиналната работа и последващи разработки. [42]

Пример:
Оптимизация на PID настройки (инженерен контрол): реални гени са K_p, K_i, K_d; fitness комбинира интеграл на квадрата на грешката (ISE) и наказания за пренастройване. Българска публикация демонстрира използване на GA за PI/PID оптимизация. [43]

Препоръчано четиво:
- Оригинален SBX (Deb & Agrawal). [44]
- Deb (1999/следващи) за self-adaptive real-parameter GA със SBX. [45]
- Статии/дискусии за mutation схеми за реални параметри. [46]

4.3 Пермутационни представяния

Navigation label: Пермутации
Slug: `/predstavyane/permutacionni/`
Какво съдържа: задачи TSP/scheduling/join ordering; защо стандартното crossover руши валидност; пермутационни crossover/mutation; repair.

Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `permutation-invalidity-example.svg` — „Защо 1-point crossover дава дубликати“ - `pmx-ox-crossover.svg` — „PMX/OX примери“ - `swap-inversion-mutation.svg` — „Swap и inversion мутация“

Обяснение:
При пермутации трябва да гарантирате, че всяко потомство остава валидна пермутация. Затова или използвате специализирани оператори (PMX/OX), или използвате индиректно кодиране (random keys). Практически пример за пермутационен/комбинаторен домейн е оптимизацията на join ordering в SQL. [26]

Ключови уравнения:
Тук уравненията са по-малко важни; по-важни са инвариантите: “всяка стойност се появява точно веднъж”. При fitness често имате цена $c(\pi)$ за пермутация π. [23]

Пример:
За пермутация $[1, 2, 3, 4, 5]$ и $[5, 4, 3, 2, 1]$, едноточково кръстосване често дава дубликати. Решение: OX — запазвате сегмент и попълвате останалите позиции по реда от другия родител. [47]

Препоръчано четиво:
- Български: Велев (СУ) за SQL join ordering. [23]
- Random keys (Bean, INFORMS) като алтернатива. [48]

4.4 Random keys и индиректно кодиране

Navigation label: Random keys
Slug: `/predstavyane/random-keys/`
Какво съдържа: принцип, декодиране чрез сортиране, защо помага за валидност; кога е подходящо; рискове (неинжективност при равни ключове).
Аудитория: Advanced
Дължина: ~2000–3500 думи
Активи: - `random-keys-decoding.svg` — „Random keys → sorting → пермутация“ - `random-keys-ties.svg` — „Как да обработим равни ключове (ties)“

Обяснение:
Random keys представя пермутация чрез реални ключове; декодерът сортира ключовете и извежда реда. Това пренася сложността (валидност) от operator-level към decoder-level и често улеснява използването на стандартни реално-кодирани оператори. Идеята е канонично формулирана от Bean (1994). [48]

Ключови уравнения:
- Декодиране: за ключове $k \in \mathbb{R}^n$, пермутация $\pi = \mathrm{argsort}(k)$. [48]

Пример:
Ключове $[0.2, 0.9, 0.1]$ → argsort дава индекси $[3, 1, 2]$ (в зависимост от индексирането), което е пермутацията. [48]

Препоръчано четиво:
- Bean (1994), INFORMS DOI страница. [48]

4.5 Дървета и Genetic Programming

Navigation label: GP
Slug: `/predstavyane/genetic-programming/`
Какво съдържа: синтактични дървета, subtree crossover/mutation, bloat, типова безопасност; връзка с GA представянията.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `gp-tree-example.svg` — „Синтактично дърво (program tree)“ - `gp-subtree-crossover.svg` — „Subtree crossover (преди/след)“ - `gp-bloat-chart.png` — „Bloat: растеж на размер на дърветата“

Обяснение:
Genetic Programming (GP) е еволюционен подход, при който индивидите са програми, най-често представени като дървета. Основният оператор е subtree crossover, който разменя поддървета между родители. MIT Press страницата за Koza (1992) е официален вход към класическата GP литература. [49]

Ключови уравнения:
Тук по-важни са ограничения/възможности на пространството от програми; добавете дефиниции за fitness като грешка на програмата върху набор от примери (напр. MSE). [50]

Пример:
Символна регресия: цел е да намерите формула (дърво), която минимизира MSE върху данни. [50]

Препоръчано четиво:
- Koza (MIT Press) — основна книга. [51]
- DEAP GP tutorial (за практическа имплементация). [52]

---

5 Оператори

Navigation label: Оператори
Slug: `/operatori/`
Какво съдържа: дизайн принципи за selection/crossover/mutation; балансиране exploration–exploitation; отделяне на parent selection от survivor selection.
Аудитория: Mixed
Дължина: ~1200–2000 думи (hub)
Активи: - `operators-overview.svg` — „Оператори и ефект върху разнообразие“ - `selection-pressure-diagram.svg` — „Селективен натиск“

5.1 Селекция

Navigation label: Селекция
Slug: `/operatori/selekciya/`
Какво съдържа: roulette/FPS, rank, tournament, SUS; селективен натиск; как да изберем метод; елитизъм като компонент на „survival“.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `roulette-wheel.svg` — „Roulette wheel selection“ - `tournament-selection.svg` — „Tournament selection“ - `sus-selection.svg` — „Stochastic universal sampling“

Обяснение:
Селекцията контролира скоростта на прогрес и риска от преждевременна сходимост. Whitley и Goldberg разглеждат селекцията като основен източник на натиск, докато вариационните оператори внасят нови комбинации. [7]

Ключови уравнения:
- Fitness-proportionate selection (рулетка):

$$p_i = \frac{F_i}{\sum_j F_j}.$$

- Турнирна селекция: параметър k (размер на турнира) контролира натиска; полезно е да включите вероятност за избор на най-добрия индивид в турнир при избор с възстановяване: $1 - (1 - 1/N)^k$. [53]

Пример:
При $N = 100$ и $k = 3$, най-добрият индивид има ~3% шанс да спечели даден турнир; това дава умерен натиск. Реалната честота като родител зависи от броя турнири/родители. [54]

Препоръчано четиво:
- Whitley tutorial (селективен натиск, варианти). [9]
- Goldberg (класическа практика и fitness scaling). [10]

5.2 Кръстосване

Navigation label: Кръстосване
Slug: `/operatori/krastosvane/`
Какво съдържа: 1-point/2-point/uniform, arithmetic crossover, SBX; при пермутации: OX/PMX; кога crossover помага/пречи; disruption на schemas.
Аудитория: Mixed
Дължина: ~3000–5000 думи
Активи: - `one-two-point-crossover.svg` — „1-point и 2-point crossover“ - `uniform-crossover.svg` — „Uniform crossover“ - `sbx-illustration.svg` — „SBX“ - `permutation-crossover-ox-pmx.svg` — „OX/PMX за пермутации“

Обяснение:
Кръстосването е механизъм за рекомбинация: комбинира „частични решения“. При бинарни низове класически са 1-point/2-point/uniform. При реални параметри SBX е утвърден оператор. При пермутации са нужни специални оператори, за да се запази валидност. [55]

Ключови уравнения:
- За SBX включете формите за y_1, y_2 и ролята на η_c (distribution index). [56]

Пример:
(Бинарно) при cut-point c: дете1 = prefix(родител1, c) + suffix(родител2, c).
(Реално) SBX: при близки родители получавате деца близо; при далечни — по-широк диапазон, което поддържа exploration. [57]

Препоръчано четиво:
- Deb & Agrawal (SBX). [44]
- Whitley tutorial (crossover и schema disruption). [9]

5.3 Мутация

Navigation label: Мутация
Slug: `/operatori/mutaciya/`
Какво съдържа: bit-flip, swap/inversion, Gaussian и polynomial mutation; mutation rate heuristics ($1/L$); адаптивна/самоадаптивна мутация.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `bitflip-mutation.svg` — „Bit-flip“ - `swap-inversion.svg` — „Swap/Inversion“ - `gaussian-vs-polynomial.svg` — „Gaussian vs polynomial mutation (интуитивно)“ - `mutation-rate-sweep.png` — „Ефект от мутационна скорост върху качество/разнообразие“

Обяснение:
Мутацията е основен механизъм за поддържане на разнообразие. При прекалено ниска мутация и силна селекция популацията може да се хомогенизира (преждевременна сходимост). При прекалено висока мутация алгоритъмът се приближава към случайно търсене. В реално-кодирани GA широко се използва полиномиална и гаусова мутация; Deb анализира mutation схеми за реални параметри. [58]

Ключови уравнения:
- Bit-flip: $x_i \leftarrow 1 - x_i$ с вероятност p_m.
- Евристика: $p_m \approx 1/L$ (средно ~1 мутация на индивид при L гени). [59]

Пример:
За $L = 100$, $p_m = 0.01$ означава ~1 flip на индивид средно; това често работи като „стартова точка“, после се настройва. [53]

Препоръчано четиво:
- Whitley tutorial (роля на мутацията). [9]
- Deb (анализ на mutation schemes за real-parameter GA). [46]

---

6 Фитнес и целеви функции

Navigation label: Фитнес
Slug: `/fitnes/`
Какво съдържа: дизайн на fitness, scaling/normalization, шум и робустност, multiobjective fitness, обработка на ограничения (penalty, Deb rules, repair).
Аудитория: Mixed
Дължина: ~1500–2500 думи (hub)

6.1 Дефиниране и скалиране

Navigation label: Скалиране
Slug: `/fitnes/skalirane/`
Какво съдържа: fitness scaling (за да се избегне доминация/стагнация), linear scaling, rank-based fitness, handling на негативни стойности, шум.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `fitness-scaling-examples.svg` — „Скалиране: суров vs скалиран fitness“ - `noise-robustness.svg` — „Шум в оценката и стабилизиране (повторения)“

Обяснение:
Fitness е интерфейс към селекцията: ако fitness разпределението стане твърде „остро“ (няколко много добри индивида), рулетката може да стане нестабилна; ако е твърде „плоско“, селекцията няма сигнал. Класическите текстове подчертават fitness scaling като практическо условие за стабилност. [60]

Ключови уравнения:
- При минимизация на цена $c(x)$ често минавате към максимизация, напр. $F(x) = \frac{1}{1+c(x)}$ (внимание към мащаби). [61]

Пример:
Ако $c(x)$ варира от 1 до 1 000 000, $F(x) = 1/(1+c)$ компресира диапазона силно; по-добре е да нормализирате или да използвате ранкова селекция. [60]

Препоръчано четиво:
- Goldberg (fitness scaling). [10]
- Попов (2003) за практически постановки и MATLAB имплементация. [62]

6.2 Ограничения

Navigation label: Ограничения
Slug: `/fitnes/ogranicheniya/`
Какво съдържа: penalty функции, Deb constraint handling (без penalty параметър), feasibility rules, repair, constrained dominance.
Аудитория: Advanced
Дължина: ~3000–5000 думи
Активи: - `constraint-penalty-vs-deb.svg` — „Penalty vs Deb rules“ - `repair-example.svg` — „Repair на infeasible решение“ - `feasible-dominance.svg` — „Feasible dominance / constrained dominance“

Обяснение:
При ограничения, ключов риск е penalty параметрите да са трудни за настройване (или убиват селективния сигнал, или пропускат feasibility). Deb (2000) предлага constraint handling в рамките на „penalty“ философията без нужда от penalty параметър, което прави метода по-практичен. В многоцелеви контекст NSGA-II също модифицира доминирането за constrained problems. [63]

Ключови уравнения:
- Penalty форма (пример):

$$F(x) = f(x) - \lambda \sum_i \max(0, g_i(x)) - \lambda \sum_j |h_j(x)|.$$

- Constrained dominance (как да опишете): feasible решения са предпочитани; между infeasible — по-малко нарушение. [64]

Пример:
В box constraints за реални параметри, repair може да е “clip”: $x_i \leftarrow \min(\max(x_i, l_i), u_i)$. Но това може да въведе bias към границите; алтернатива е reflect. [56]

Препоръчано четиво:
- Deb (2000) за constraint handling. [65]
- NSGA-II (constrained dominance). [66]

---

7 Параметри и динамика

Navigation label: Параметри
Slug: `/parametri/`
Какво съдържа: параметри на GA, tuning/control, диагностични метрики за разнообразие, island models, преждевременна конвергенция.
Аудитория: Mixed
Дължина: ~1500–2500 думи (hub)

7.1 Настройване и контрол на параметри

Navigation label: Tuning & control
Slug: `/parametri/tuning-control/`
Какво съдържа: разлика tuning vs control; детерминистичен, адаптивен и самоадаптивен контрол; кога какво е оправдано; експериментален дизайн за tuning.
Аудитория: Advanced
Дължина: ~3000–5000 думи
Активи: - `parameter-control-taxonomy.svg` — „Класификация на parameter control“ - `adaptive-mutation-schedule.svg` — „Примерна адаптивна схема“

Обяснение:
Контролът на параметри е систематизиран като ключова област: вместо да фиксирате p_c, p_m, N предварително, можете да ги изменяте по време на оптимизацията според показатели (diversity, прогрес). Класическите обзори дават терминология и класификация (tuning vs deterministic/adaptive/self-adaptive). [67]

Ключови уравнения:
- Пример за детерминистичен schedule: $p_m(t) = p_{m,0} \cdot \alpha^t$ (илюстративно; страницата трябва да подчертае, че това е design choice и се валидира експериментално). [54]

Пример:
Адаптивен контрол: ако diversity падне под праг, увеличете p_m или намалете селективния натиск (напр. по-малък tournament size). [68]

Препоръчано четиво:
- Eiben et al. (Parameter Control in Evolutionary Algorithms). [69]
- Whitley (практически последици). [9]

7.2 Сходимост и преждевременна конвергенция

Navigation label: Сходимост и разнообразие
Slug: `/parametri/sxodimost-diversitet/`
Какво съдържа: как да измерим конвергенция; как да засечем преждевременна сходимост; tradeoff; връзка с оператори.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `convergence-plot-template.png` — „Best/mean fitness по поколения“ - `diversity-heterozygosity-plot.png` — „Хетерозиготност по поколения“ - `premature-convergence-signals.svg` — „Сигнали за преждевременна сходимост“

Обяснение:
В практиката конвергенцията е наблюдавано насищане на best/mean fitness, често съпроводено със спад на разнообразието. Ако това стане рано и далеч от оптимум, имате преждевременна сходимост. Whitley разглежда разнообразието и island/cellular модели като антидот. [70]

Ключови уравнения:
- Проста мярка за diversity при бинарни гени: средна хетерозиготност

$$D = \frac{1}{L}\sum_{i=1}^{L} 2p_i(1-p_i),$$

където p_i е честотата на “1” в позиция i. [9]

Пример:
Ако $D \to 0$, популацията е почти идентична; повишете мутацията, въведете island migration, или приложете niching. [71]

Препоръчано четиво:
- Whitley tutorial (diversity, island GA). [9]
- ESA pagmo/pygmo (island model като интерфейс). [72]

7.3 Модели за разнообразие

Navigation label: Механизми
Slug: `/parametri/diversitet-mehanizmi/`
Какво съдържа: niching (sharing/crowding), island model, cellular GA; миграционни политики и параметри; кога какво е подходящо.
Аудитория: Advanced
Дължина: ~2500–4500 думи
Активи: - `island-model-migration.svg` — „Острови и миграция“ - `niching-sharing.svg` — „Fitness sharing“ - `crowding-diagram.svg` — „Crowding replacement“

Обяснение:
Разнообразието може да се „инжектира“ структурно (острови, локални съседи) или чрез fitness/замяна (sharing/crowding). Island моделите са особено ценни при паралелизация и скъпи оценки; pagmo/pygmo е официален open-source пример за такава абстракция. [73]

Ключови уравнения:
- Fitness sharing: $F_i' = \frac{F_i}{\sum_j sh(d(i,j))}$ с sharing функция $sh(\cdot)$ и праг σ (страницата трябва да дефинира sh и метрика за дистанция). [74]

Пример:
Острови: 8 острова по 50 индивида, миграция на 1–2 елитни/случайни индивида на всеки 20 поколения; цел — да ограничите доминацията на една линия и да изследвате паралелно различни региони. [75]

Препоръчано четиво:
- Whitley tutorial (island/cellular GA). [9]
- pagmo/pygmo документация (island abstraction). [76]

---

8 Разширени методи

Navigation label: Разширения
Slug: `/razshireniya/`
Какво съдържа: меметични/хибридни методи; многоцелеви оптимизации; обработка на ограничения в multiobjective контекст.
Аудитория: Mixed
Дължина: ~1200–2000 думи (hub)

8.1 Хибридни и меметични алгоритми

Navigation label: Меметични
Slug: `/razshireniya/memetichni/`
Какво съдържа: идея GA + локално търсене; стратегии кога да се прилага локално търсене; бюджетиране; рискове (over-exploitation).
Аудитория: Advanced
Дължина: ~2500–4500 думи
Активи: - `memetic-loop.svg` — „GA + local search (след вариация)“ - `global-vs-local-budget.svg` — „Разпределение на бюджет между глобално и локално търсене“

Обяснение:
Хибридизацията е практическа техника за ускоряване: GA осигурява глобално изследване, локалното търсене дооптимизира отделни индивиди. Това е типична „инженерна“ еволюция на GA към по-ефективни алгоритми при скъпи задачи. [77]

Ключови уравнения:
Няма една универсална формула; страницата трябва да включи „бюджет“: $B = B_{\mathrm{GA}} + B_{\mathrm{LS}}$ и емпирични правила за избор на част от индивидите за локално търсене. [17]

Пример:
GA генерира 100 потомци; върху най-добрите 10 се правят 50 стъпки локално търсене; после се връщат в популацията. [53]

Препоръчано четиво:
- (Като „primary-like“ вход) Whitley: дискусия за варианти и island/cellular. [9]
- Български: Велев (комбинаторен проблем, домейн знания, методология). [23]

8.2 Многоцелеви ГА

Navigation label: Многоцелеви
Slug: `/razshireniya/multiobjective/`
Какво съдържа: Pareto доминиране, фронт, crowding, hypervolume; защо MOEA са естествени “популационни” методи; обзор на NSGA-II/III и др.
Аудитория: Mixed
Дължина: ~2000–3500 думи
Активи: - `pareto-front-illustration.png` — „Pareto front и dominance“ - `crowding-`

`distance.svg` — „Crowding distance (интуиция)“ - `hypervolume-illustration.svg` — „Hypervolume индикатор“

Обяснение:
MOEA използват популации, за да апроксимира множество от недоминирани решения (компромиси). NSGA-II е каноничен алгоритъм за многоцелеви оптимизации с nondominated sorting и crowding distance за diversity. [78]

Ключови уравнения:
- Pareto доминиране при минимизация: $a \prec b$ ако $\forall m : f_m(a) \leq f_m(b)$ и $\exists m : f_m(a) < f_m(b)$. [64]

Пример:
Дизайн на система с две цели: минимизация на маса и максимизация на якост (или минимизация на отрицателна якост). GA търси набор от решения, които показват tradeoff. [79]

Препоръчано четиво:
- Deb et al. (NSGA-II, 2002). [80]
- Zitzler et al. (performance assessment на MO optimizers). [81]
- Deb (MOEA book / intro report). [82]

8.2.1 NSGA-II

Navigation label: NSGA-II
Slug: `/razshireniya/multiobjective/nsga-ii/`
Какво съдържа: алгоритъм стъпка-по-стъпка, fast nondominated sorting, crowding distance, elitism; сложност; constrained dominance модификация; типични параметри.
Аудитория: Advanced
Дължина: ~3500–6000 думи
Активи: - `nsga2-flow.svg` — „NSGA-II: mating + survival“ - `fast-nondominated-sorting.svg` — „Fast nondominated sorting (схема)“ - `crowding-distance-plot.png` — „Crowding distance: визуализация“

Обяснение:
NSGA-II решава основни критики към ранни nondominated sorting + sharing подходи: (1) висока изчислителна сложност, (2) липса на елитизъм, (3) нужда от sharing parameter, като въвежда crowding distance и елитистко оцеляване. [66]

Ключови уравнения:
- Crowding distance (какво да включи): сумарна нормализирана дистанция в целевото пространство към съседите във фронта (опишете формално по Deb 2002). [80]
- Сложност: типично се обсъжда $O(MN^2)$ за сортиране по недоминираност (M цели, N популация) в описанията на алгоритъма. [66]

Пример:
Разделете популацията на фронтове $F_1, F_2, \ldots$ (недоминирани, доминирани веднъж, ...). Запълнете новата популация фронт по фронт; ако последният фронт не се побира, изберете тези с по-голям crowding distance. [83]

Препоръчано четиво:
- Оригинален NSGA-II PDF (Deb et al.). [80]
- Zitzler et al. (метрики/оценяване). [81]
- Pymoo NSGA-II страница (референция към оригинала + имплементационни бележки). [84]

8.2.2 NSGA-III

Navigation label: NSGA-III
Slug: `/razshireniya/multiobjective/nsga-iii/`
Какво съдържа: мотиви (many-objective), reference points, selection механизъм; разлики спрямо NSGA-II; практични настройки.
Аудитория: Advanced
Дължина: ~3000–5500 думи
Активи: - `nsga3-reference-points.svg` — „Reference points и асоцииране“ - `many-objective-crowding-issue.svg` — „Защо crowding distance деградира“

Обяснение:
При много цели (many-objective) crowding distance става по-малко дискриминативен. NSGA-III използва reference points за поддържане на равномерно покритие. [85]

Ключови уравнения:
Тук по-скоро се изисква формално описание на асоцииране на решения към reference directions и selection на база „нишови“ counts; страницата следва да цитира оригиналния доклад/публикации на Deb. [86]

Пример:
При 10 цели избирате reference points в simplex и поддържате решения около тях, за да избегнете концентрация в малък регион. [86]

Препоръчано четиво:
- Deb (KanGAL report / NSGA-III материал). [86]
- Zitzler et al. (метрики/измерване на качество). [81]

8.2.3 MOEA/D, SPEA2, IBEA

Navigation label: Други MOEA
Slug: `/razshireniya/multiobjective/drugi-moea/`
Какво съдържа: декомпозиционни (MOEA/D), strength-based (SPEA2), indicator-based (IBEA); кога тези са по-подходящи; сравнение.
Аудитория: Advanced
Дължина: ~3000–5500 думи
Активи: - `moead-decomposition.svg` — „MOEA/D: разлагане на MOO“ - `spea2-archive.svg` — „SPEA2 архив и density“ - `ibea-indicator.svg` — „Indicator-based selection“

Обяснение:
Различните MOEA се различават по това „как“ оценяват и подбират решения: NSGA-II е dominance + diversity, MOEA/D е decomposition, SPEA2 разчита на strength и архив, IBEA използва индикатор като

hypervolume за селекция. Това е важно за дизайн при специфични изисквания (many objectives, constraints, предпочитания). [87]

Ключови уравнения:
- Hypervolume индикатор (концептуална дефиниция като мярка на обединение на доминираните хиперправоъгълници към референтна точка). [88]

Пример:
MOEA/D: всяка подзадача има weight vector; поддържате neighborhood; обменяте решения локално. [89]

Препоръчано четиво:
- Zitzler et al. (performance assessment и индикатори). [81]
- Auger et al. (theory of hypervolume indicator). [90]
- Platypus списък на поддържани MOEA (като практичен ориентир). [91]

---

9 Оценяване и бенчмаркове

Navigation label: Оценяване
Slug: `/ocenyavane/`
Какво съдържа: метрики, бенчмаркове, експериментални протоколи, сложност/бюджет.
Аудитория: Mixed
Дължина: ~1200–2000 думи (hub)

9.1 Метрики

Navigation label: Метрики
Slug: `/ocenyavane/metriki/`
Какво съдържа: разделяне single- vs multi-objective; защо “fitness” ≠ “benchmark metric”; статистически сравнения.
Аудитория: Mixed
Дължина: ~1500–2500 думи
Активи: - `metrics-map.svg` — „Метрики за GA/MOEA“ - `boxplots-experiment-template.png` — „Boxplot за множество стартирания“

9.1.1 Едноцелеви метрики

Navigation label: Едноцелеви
Slug: `/ocenyavane/metriki/ednocel/`
Какво съдържа: best-so-far, mean/median, success rate, runtime в оценки; ERT (expected running time) и ECDF.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `ert-definition.svg` — „ERT формула и интерпретация“ - `ecdf-runtime.svg` — „ECDF на runtime (пример)“

Обяснение:
В black-box benchmarking стандартно измервате време не в секунди, а в **брой оценки на целевата функция**, защото това е най-скъпата операция. COCO/BBOB използва ERT като ключова метрика: очакван брой оценки за достигане на target за първи път (със счетоводство на успехи/неуспехи). [92]

Ключови уравнения:
ERT формула (как да се опише в страницата): очакван брой function evaluations до достигане на target при множествени независими опити; Hansen et al. описват ERT в bbob experimental setup документа. [31]

Пример:
30 независими старта; цел е $f(x) \leq t$. Ако 20 старта успяват, ERT отчита сумарните оценки (вкл. на неуспешните до budget) разделено на броя успехи. [31]

Препоръчано четиво:
- COCO/BBOB test suite page. [93]
- Hansen bbob experiment документа (ERT). [31]

9.1.2 Многоцелеви метрики

Navigation label: Многоцелеви
Slug: `/ocenyavane/metriki/mnogocel/`
Какво съдържа: hypervolume, IGD/epsilon, spread/diversity; pitfalls (референтна точка, скалиране); статистика.
Аудитория: Advanced
Дължина: ~3000–5000 думи
Активи: - `hypervolume-reference-point.svg` — „Избор на референтна точка“ - `igd-illustration.svg` — „IGD: идея“ - `pareto-coverage-metrics.svg` — „Coverage/spread“

Обяснение:
Многоцелевите метрики са отделна дисциплина: трябва да измерят едновременно конвергенция към фронта и простиране/покритие. Zitzler et al. дават анализ и обзор на performance assessment за multiobjective optimizers, а Auger et al. разглеждат theory на hypervolume индикатора и влиянието на референтната точка. [94]

Ключови уравнения:
- Hypervolume: мярка на обединението от доминирани хиперправоъгълници към референтна точка. [88]

Пример:
Два алгоритъма могат да имат еднакъв hypervolume при различни разпределения на решения; затова често комбинирате HV с spread/spacing или с множество targets. [94]

Препоръчано четиво:
- Zitzler et al. (2003) performance assessment. [81]
- Auger et al. (hypervolume theory). [90]

9.2 Бенчмаркове COCO/BBOB

Navigation label: COCO/BBOB
Slug: `/ocenyavane/benchmarks/coco-bbob/`
Какво съдържа: какво е COCO; BBOB suite (24 функции, размерности/инстанции), как се репортва (ECDF, ERT); как да сравняваме GA с други деривативно-независими методи.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `bbob-24-functions-table.png` — „Таблица: функции в BBOB“ - `ecdf-example.png` — „ECDF извадка“ - `bbob-experiment-protocol.svg` — „Протокол: budgets, targets“

Обяснение:
BBOB е стандартна test suite в COCO с 24 безшумни, едноцелеви, мащабируеми функции и множество размерности/инстанции; COCO предоставя инфраструктура за експерименти и постпроцесинг, като ERT е ключова метрика. [36]

Ключови уравнения:
- ERT (виж метрики). [31]

Пример:
Сравнявате две конфигурации на GA (различни p_m, N) върху 24 функции в размерност 10 и гледате ECDF на runtime за targets. [36]

Препоръчано четиво:
- BBOB suite страница. [93]
- Hansen bbob experiment doc (подробен setup). [18]

9.3 ZDT, DTLZ, WFG

Navigation label: ZDT/DTLZ/WFG
Slug: `/ocenyavane/benchmarks/zdt-dtlz-wfg/`
Какво съдържа: ZDT функции (трудности: мултимодалност/deception), DTLZ скалируеми тестове, WFG toolkit (bias/multimodality/nonseparability), кога кой се използва.
Аудитория: Advanced
Дължина: ~3500–6000 думи
Активи: - `zdt-front-shapes.png` — „Форми на Pareto фронтове (ZDT)“ - `dtlz-scalability.svg` — „DTLZ: скалируемост по цели/променливи“ - `wfg-transformations.svg` — „WFG transformations: bias, multimodality, non-separability“

Обяснение:
Zitzler–Deb–Thiele предлагат емпирично сравнение на MOEA върху набор тестови функции (ZDT). DTLZ предоставя скалируеми multiobjective тестове. WFG toolkit систематизира генерация на тестови проблеми с контролируеми свойства и е широко използван benchmark за MOEA. [95]

Ключови уравнения:
Тук е уместно да се дадат конкретни дефиниции за 1–2 представителни функции (напр. ZDT1, DTLZ2) и да се обясни как се конструира Pareto фронтът. Източниците са ZDT статията и DTLZ материалите. [96]

Пример:
WFG: създавате проблеми, които са едновременно non-separable и multimodal, за да проверите robustността на MOEA спрямо „реалистични“ трудности. [97]

Препоръчано четиво:
- Zitzler–Deb–Thiele (2000) (ZDT-based empirical results). [98]
- DTLZ скалируеми тестове (ETHZ PDF). [99]
- Huband et al. (WFG toolkit, TEVC 2006). [100]

9.4 Сложност и бюджет на оценки

Navigation label: Сложност
Slug: `/ocenyavane/slojnost/`
Какво съдържа: сложност на GA на поколение; доминиращата цена е fitness evaluation; NSGA-II сортиране; практическо бюджетиране и паралелизация.
Аудитория: Mixed
Дължина: ~2000–3500 думи
Активи: - `complexity-breakdown.svg` — „Разбивка на цена: eval vs operators“ - `nsga2-complexity.svg` — „O(MN^2) (контекст)“ - `parallel-islands-benefit.svg` — „Паралелизация чрез острови“

Обяснение:
За много задачи основната цена е оценката на целевата функция; затова измерването в #evaluations и паралелизацията (островни модели) са ключови. При NSGA-II има допълнителен разход за nondominated sorting, който е важен при големи популации. [101]

Ключови уравнения:
- Generational GA: оценка $\sim O(N \cdot C_{\text{eval}})$, вариация $\sim O(NL)$.
- NSGA-II: обсъждана сложност за nondominated sorting $\sim O(MN^2)$. [66]

Пример:
При скъпа симулация C_{eval} доминира, и най-важният дизайн избор е как да редуцирате оценки (surrogates, reuse, parallel). При евтини оценки — оптимизирайте сортировките и операторите. [102]

Препоръчано четиво:
- NSGA-II (discussion на complexity). [80]
- COCO/BBOB (evaluation-budget ориентирана методология). [17]

---

10 Имплементация и инструменти

Navigation label: Имплементация
Slug: `/implementaciya/`
Какво съдържа: псевдокод, шаблони, debugging, reproducibility; библиотеки (Python/Java/...); препоръки за open-source.

Аудитория: Mixed
Дължина: ~1200–2000 думи (hub)

10.1 Псевдокод и шаблони

Navigation label: Псевдокод
Slug: `/implementaciya/psevdokod/`
Какво съдържа: каноничен GA псевдокод, steady-state вариант, NSGA-II skeleton; „какво да логваме“.
Аудитория: Beginner
Дължина: ~2000–3500 думи
Активи: - `ga-pseudocode-blocks.svg` — „Блокове: init/eval/select/variation/replace“ - `logging-checklist.svg` — „Checklist: какво се логва“

Псевдокод (пример; коментари на български):

```
# Псевдокод: Каноничен генетичен алгоритъм (generational) с елитизъм
# Вход: decode(phi), fitness(f), N (популация), pc (кръстосване), pm (мутация),
#       E (брой елит), stop() (критерий за спиране)

def genetic_algorithm():
    P = init_population(N)          # случайна/хибридна инициализация
    F = evaluate(P)                 # F[i] = f(phi(P[i]))

    while not stop(P, F):
        elite = select_best(P, F, E)

        parents = parent_selection(P, F)    # напр. tournament, SUS
        offspring = []

        while len(offspring) < N:
            p1, p2 = pick_two(parents)

            if rand() < pc:
                c1, c2 = crossover(p1, p2) # внимателно според представянето
            else:
                c1, c2 = copy(p1), copy(p2)

            c1 = mutate(c1, pm)
            c2 = mutate(c2, pm)

            offspring.append(c1)
            offspring.append(c2)

        F_off = evaluate(offspring)

        P, F = survivor_selection(offspring, F_off, elite)  # елитизъм/замяна
```

```
    return best(P, F)
```

Какво да обясни страницата:
Този псевдокод е „минималният“ скелет; реалните системи добавят repair, ограничителни правила, паралелизация и протокол за повторяемост. Практически референции са примерите в DEAP (напр. OneMax) и общата рамка в Whitley. [103]

Ключови уравнения:
- Връзка бюджет–параметри: $B \approx N \times G$ (при generational за G поколения). [17]

Пример:
Ако имате ограничения, `mutate()` трябва да е “constraint-aware” (или след мутация `repair()`), иначе fitness може да стане доминиран от penalties. [104]

Препоръчано четиво:
- DEAP OneMax пример. [39]
- Whitley tutorial. [9]

10.2 Библиотеки и tooling

Navigation label: Библиотеки
Slug: `/implementaciya/biblioteki/`
Какво съдържа: сравнение на библиотеки, кога коя да ползвате; минимални примерни рецепти; как да структурирате код.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `libraries-comparison-table.png` — „Сравнение на библиотеки“ - `library-selection-flow.svg` — „Как да изберем библиотека“

Обяснение:
Използването на утвърдени библиотеки намалява риска от грешки в оператора и улеснява benchmarking. DEAP е framework за rapid prototyping с прозрачен модел на алгоритмите; PyGAD е насочен към лесна употреба и ML интеграция; pygmo/pagmo предлага island-model паралелизация; pymoo и Platypus имат силни MOEA възможности; в Java екосистемата jMetal, MOEA Framework, ECJ и Jenetics са често използвани. [105]

Ключови уравнения:
Няма нужда от нови формули; страницата трябва да фокусира API-концепции: индивид, fitness, оператори, logbook/статистика. [106]

Пример:
DEAP позволява да дефинирате `toolbox.select`, `toolbox.mate`, `toolbox.mutate`; pymoo предоставя готови реализации на NSGA-II и метрики за анализ. [107]

Препоръчано четиво:
- DEAP docs. [108]
- PyGAD docs. [109]
- ESA pagmo/pygmo. [110]
- pymoo paper (Blank & Deb). [111]
- jMetal docs. [112]
- MOEA Framework сайт. [113]
- Jenetics manual. [114]

10.3 Повторяемост и експериментален протокол

Navigation label: Reproducibility
Slug: `/implementaciya/reproducibility/`
Какво съдържа: seed management, многократни стартирания, статистически тестове, протокол за benchmarking; логове и артефакти.
Аудитория: Advanced
Дължина: ~2000–3500 думи
Активи: - `repro-checklist.svg` — „Checklist: seeds, versions, configs“ - `experiment-folder-structure.svg` — „Структура на експериментална папка“ - `report-template.md` — „Шаблон за репорт“

Обяснение:
Стохастичните методи изискват множествени независими стартирания и стандартни метрики. COCO/BBOB документира експериментални процедури и начини на представяне. Страницата трябва да изисква запис на параметри, версия на кода, seed и условия за спиране. [92]

Ключови уравнения:
- ERT и success rate са ключови за обобщение на множествени runs. [31]

Пример:
„Изчисляваме средно качество“ е недостатъчно: при тежки опашки е по-устойчив median/quantiles; при benchmarking се използват target-based метрики като ERT. [115]

Препоръчано четиво:
- Hansen bbob experiment doc. [31]
- Zitzler et al. (multiobjective performance assessment). [81]

---

11 Приложения и казуси

Navigation label: Приложения
Slug: `/prilozheniya/`
Какво съдържа: типове приложения, избор на GA варианти по домейн, кратки case studies.
Аудитория: Mixed
Дължина: ~1500–2500 думи (hub)

11.1 Управление и инженерна оптимизация

Navigation label: Инженерство
Slug: `/prilozheniya/inzhenerstvo-upravlenie/`
Какво съдържа: реално-кодирани параметри, multiobjective design, ограничения; пример PID/fuzzy controllers, структуриране на fitness.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `pid-ga-case-architecture.svg` — „GA за PID tuning: архитектура“ - `tradeoff-curve.png` — „Компромис точност–робастност“

Обяснение:
GA се използва за offline настройка на параметри на регулатори и оптимизация на критерии за качество на управление. Българска публикация описва GA-базирана оптимизация на PI/PID и критерии на база интеграли от грешка. [116]

Ключови уравнения:
- ISE: $\mathrm{ISE} = \int_0^T e(t)^2\,dt$ (или дискретен аналог). Страницата трябва да даде как се комбинира с penalties за overshoot/settling. [43]

Пример:
Фитнес = $-(\mathrm{ISE} + \lambda \cdot \mathrm{Overshoot})$, оптимизирано чрез реално-кодиран GA със SBX и polynomial mutation. [117]

Препоръчано четиво:
- Българска статия за GA-базирана оптимизация на регулатори. [43]
- SBX източник. [44]

11.2 Оптимизация на SQL заявки

Navigation label: SQL
Slug: `/prilozheniya/sql/`
Какво съдържа: join ordering като комбинаторна оптимизация; представяне (пермутации/дървета), fitness (cost model), ограничения и практическа методология.
Аудитория: Advanced
Дължина: ~3000–5000 думи
Активи: - `sql-join-ordering-ga.svg` — „GA за join ordering (pipeline)“ - `permutation-encoding-sql.svg` — „Кодиране на join ред“ - `case-study-results-table.png` — „Таблица: качество/време по конфигурации“

Обяснение:
Join ordering е класически труден проблем в оптимизация на заявки; подходи с GA могат да се използват за пространство на планове, когато точните алгоритми са непрактични при много join-ове. Български автореферат (СУ) разглежда GA за оптимизация на SQL заявки и дава методологични и експериментални аспекти. [118]

Ключови уравнения:
- Fitness често е минус на cost: $F(\pi) = -c(\pi)$ или $F(\pi) = 1/(1 + c(\pi))$, където c е оценка на план. [119]

Пример:
Пермутационен GA с OX crossover + swap/inversion mutation, плюс repair за валидност на планове. [120]

Препоръчано четиво:
- Велев (СУ) — автореферат. [23]
- Random keys (Bean) като алтернативна стратегия за sequencing problems. [48]

11.3 Генетични размити системи

Navigation label: Размити системи
Slug: `/prilozheniya/genetichni-razmiti-sistemi/`
Какво съдържа: GA за настройка на fuzzy rule bases и параметри; хибридизация на soft computing; типове цели/ограничения.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `ga-fuzzy-hybrid.svg` — „GA + fuzzy: какво оптимизираме“ - `rulebase-encoding.svg` — „Кодиране на rules/параметри“

Обяснение:
Генетичните размити системи комбинират размита логика и GA за оптимизация на правила/параметри; български монографичен материал е наличен като резюмета/описания и може да бъде ползван като локален ресурс. [121]

Ключови уравнения:
Fitness често комбинира точност на моделиране и сложност на rule base (penalty за брой правила). [122]

Пример:
Оптимизация на membership function параметри чрез реално-кодиран GA; penalty за прекалено много правила. [123]

Препоръчано четиво:
- Български материал (БСУ резюмета). [122]
- Попов (2003) като GA фундамент на български. [16]

---

12 Чести капани и добри практики

Navigation label: Капани
Slug: `/kapani/`
Какво съдържа: списък от „антипатерни“, диагностични тестове, препоръчани протоколи за оценяване, checklist-и.

Аудитория: Mixed
Дължина: ~1200–2000 думи (hub)

12.1 Типични грешки при дизайн

Navigation label: Дизайн
Slug: `/kapani/dizayn/`
Какво съдържа: грешно представяне; неподходящи оператори; прекомерен/недостатъчен селективен натиск; penalty капани; липса на diversity.
Аудитория: Mixed
Дължина: ~2500–4500 думи
Активи: - `pitfalls-representation.svg` — „Representation–operator mismatch“ - `selection-pressure-too-high.svg` — „Твърде силен натиск“ - `penalty-trap.svg` — „Penalty доминира fitness“

Обяснение:
Най-честите системни провали са: (1) mismatch между представяне и crossover (невалидни решения), (2) fitness/penalty скалиране, което убива сигнала, (3) преждевременна сходимост поради липса на diversity механизми. Това следва директно от каноничните дискусии за operator disruption и parameter control. [124]

Ключови уравнения:
- Напомнете формата на penalty и защо λ е труден (и покажете Deb-style алтернатива). [65]

Пример:
Пермутационна задача + едноточково crossover → дубликати → repair, което вкарва bias и може да убие ефекта на crossover. По-добре е да смените представянето (random keys) или оператора (OX/PMX). [26]

Препоръчано четиво:
- Whitley tutorial (оператори и pitfalls). [9]
- Bean (random keys) като remedy за sequencing. [48]
- Deb (constraints). [65]

12.2 Типични грешки при оценяване

Navigation label: Оценяване
Slug: `/kapani/ocenyavane/`
Какво съдържа: single-run сравнения; липса на seeds; измерване в секунди вместо #eval; непълни метрики при MOEA; неправилен избор на reference point за hypervolume; cherry-picking.
Аудитория: Advanced
Дължина: ~2500–4500 думи
Активи: - `evaluation-pitfalls-checklist.svg` — „Checklist за експерименти“ - `hypervolume-reference-pitfall.svg` — „Референтна точка: капан“

Обяснение:
За коректно сравнение на стохастични оптимизатори е нужен протокол: множество независими

старта, таргет-базирани метрики (напр. ERT), и ясни условия на спиране. При многоцелеви оценки метриките и техните параметри (референтна точка) могат да обърнат изводите. COCO/BBOB и работите на Zitzler/Thiele за performance assessment са основа за дисциплинирано оценяване. [125]

Ключови уравнения:
- ERT (вж. 9.1.1). [31]
- Hypervolume зависимост от reference point. [88]

Пример:
Да репортнете само “best result” от един run е статистически слаб аргумент; вместо това използвайте разпределение от runs и target-based метрики. [115]

Препоръчано четиво:
- Hansen bbob experiment doc. [31]
- Zitzler et al. (2003). [81]
- Auger et al. (hypervolume). [90]

---

13 Източници

Navigation label: Източници
Slug: `/iztochnici/`
Какво съдържа: библиография, подредена по приоритет: основополагащи монографии, ключови статии, benchmarking/метрики, библиотеки, български ресурси; препоръка за „пътеки на четене“ (beginner → advanced).
Аудитория: Mixed
Дължина: ~1200–2500 думи
Активи: - `reading-paths.svg` — „Пътеки за четене: beginner/mixed/advanced“

Приоритизирани първични/официални източници (с линкове чрез цитати):

- John Holland [126]; Adaptation in Natural and Artificial Systems [127] — фундаментален текст за адаптация и основи на GA традицията. [15]
- David E. Goldberg [128]; Genetic Algorithms in Search, Optimization, and Machine Learning [129] — класическа книга за оператори, fitness scaling и практика (ACM record). [10]
- Michael D. Vose [130]; The Simple Genetic Algorithm: Foundations and Theory [131] — строг теоретичен подход към SGA. [132]
- Darrell Whitley [133] — туториал за каноничен GA, оператори, island/cellular модели, schema theorem контекст. [134]
- Kalyanmoy Deb [135] — NSGA-II (оригинална публикация) и материали за многоцелеви оптимизации. [136]
- David H. Wolpert [137] и William G. Macready [138] — No Free Lunch Theorems for Optimization. [20]
- DTLZ скалируеми тестови проблеми (ETHZ PDF). [99]
- Zitzler–Deb–Thiele (2000) и Zitzler et al. (2003) за тестови проблеми и performance assessment при MOEA. [139]
- Huband et al. (WFG toolkit, TEVC 2006) и документации, които описват WFG suite. [97]
- COCO/BBOB основна документация и ERT протокол (Hansen). [140]

- Bean (1994) — random keys за sequencing и optimization (INFORMS DOI). [48]
- Paramater control обзори: Eiben et al. [67]
- Bulgarian sources: Попов (2003) — GA за оптимизация (ТУ–София); Велев — SQL оптимизация с GA (СУ). [141]

Официални/отворени инструменти (open-source implementations):
- DEAP [142] (docs + GitHub). [143]
- PyGAD [144] (docs + GitHub). [145]
- pagmo/pygmo [146] (ESA). [147]
- pymoo [148] (site + paper). [149]
- Platypus [150] (docs). [151]
- jMetal [152] (docs). [112]
- MOEA Framework [153] (официален сайт). [154]
- ECJ [155] (GitHub). [156]
- Jenetics [157] (официален сайт + manual). [158]

Таблици за сравнение и пресети

Този раздел е предназначен да бъде copy-paste като отделни „таблични компоненти“ в сайта (или като Markdown таблици). Четирите таблици са „задължителните“, които поискахте: алгоритми/варианти, представяния, библиотеки, параметърни пресети. [159]

Таблица: сравнение на ключови GA/MOEA варианти

Подход	Къде е най-подходящ	Основна идея	Ключови рискове	Основни източници
Каноничен GA	общи дискретни/комбинаторни задачи	селекция + crossover + mutation	преждевременна сходимост; mismatch представяне–оператори	Whitley; Goldberg [7]
Real-coded GA (SBX)	инженерни непрекъснати параметри	SBX + real mutation	граници/repair bias; tuning на η	Deb & Agrawal (SBX) [56]
Random keys GA	sequencing/scheduling	индиректно кодиране с декодер	ties, декодер bias	Bean 1994 [48]
NSGA-II	2–3+ цели, нужда от Pareto набор	nondominated sorting + crowding	$O(MN^2)$; избор на параметри	Deb 2002 [66]
NSGA-III	many-objective	reference points	настройка/покритие	Deb materials [86]

Подход	Къде е най-подходящ	Основна идея	Ключови рискове	Основни източници
Island model GA	скъпи оценки/ паралелизация	субпопулации + миграция	избор на миграция; комуникационни overhead	Whitley; ESA pagmo/ pygmo [160]

Таблица: сравнение на представяния и кодирания

Представяне	Типични задачи	Типични оператори	„Добра практика“
Бинарно	feature selection, дискретни решения	1/2-point, uniform; bit-flip	започнете с $p_m \approx 1/L$; следете diversity [9]
Реално	инженерни параметри	SBX; polynomial/ Gaussian mutation	контрол на box constraints; настройка η_c [117]
Пермутации	sequencing, join ordering	OX/PMX; swap/ inversion	избягвайте naïve 1-point; обмислете random keys [26]
Дървета (GP)	програми/формули	subtree crossover/ mutation	контрол на bloat; типова дисциплина [50]

Таблица: сравнение на библиотеки/инструменти

Език	Библиотека	Силни страни	Най-добра употреба	Официален източник
Python	DEAP	прозрачен framework, prototyping, паралелизация	custom GA/GP експерименти	Docs [108]
Python	PyGAD	лесна употреба, ML интеграции	бързи приложения/ оптимизация	Docs [161]
Python/ C++	pagmo/ pygmo	island model, масивна паралелност	скъпи оценки/ паралелни експерименти	ESA [110]
Python	pymoo	MOEA, метрики, визуализация	NSGA-II/III, constrained MOO	Site/paper [149]
Python	Platypus	MOEA фокус, широка поддръжка	MOEA prototyping	Docs [151]
Java	jMetal	MOEA + indicators + experiments	академични сравнения/пакети	Docs [112]
Java	MOEA Framework	MOEA библиотека	бързи MOEA експерименти	Site [113]

Език	Библиотека	Силни страни	Най-добра употреба	Официален източник
Java	ECJ	heavyweight експерименти, GP	големи експерименти	GitHub [156]
Java	Jenetics	модерен дизайн, manual	production-ориентирани GA	Site/manual [158]

Таблица: параметърни пресети (стартови точки)

Сценарий	N	p_c	p_m	Селекция	Бележки
Учебен бинарен GA (OneMax)	50–200	0.8–0.95	$\approx 1/L$	tournament $k = 2..4$	лесно за диагностика; следете diversity [8]
Реално-кодиран инженеринг (SBX)	100–300	0.9	$\approx 1/L$ (на променлива)	tournament/ранк	настройте η_c, box constraints repair [117]
NSGA-II (общо)	100–400	0.9	$\approx 1/L$	бинарен турнир по rank+crowding	оценявайте с HV/IGD; контролирайте сортиране [162]
Пермутационни задачи	100–500	0.8–0.95	swap rate ~ $1/L$	tournament	използвайте OX/PMX или random keys [26]

Манифест на нужни диаграми и изображения

Това е **единна спецификация на активи** (без вграждане в страниците), която може да се използва за задача към дизайнер/технически автор. Файловете са предложени като именуване; формати: SVG за схеми/диаграми и PNG за графики/plots.

Файл	Капшън (надпис)	Къде се използва
`site-navigation.svg`	„Карта на сайта: Генетични алгоритми“	`/`
`ga-process-flow.svg`	„Каноничен процес на генетичен алгоритъм“	`/ga/kanonichen-proces/`
`fitness-landscape-intro.png`	„Фитнес ландшафт: локални и глобални оптимуми“	`/ga/`
`schema-example.svg`	„Schema над {0,1,*}: ред и дефинираща дължина“	`/teoriya/schema-theorem/`

Файл	Капшън (надпис)	Къде се използва
`markov-chain-ga.svg`	„GA като Марков процес върху популации“	`/teoriya/sxodimost/`
`roulette-wheel.svg`	„Fitness-proportionate (roulette wheel) selection“	`/operatori/selekciya/`
`tournament-selection.svg`	„Tournament selection: параметър k и натиск“	`/operatori/selekciya/`
`one-two-point-crossover.svg`	„1-point/2-point crossover“	`/operatori/krastosvane/`
`uniform-crossover.svg`	„Uniform crossover“	`/operatori/krastosvane/`
`sbx-illustration.svg`	„SBX: генериране на деца около родителите“	`/predstavyane/realno-kodirani/`, `/operatori/krastosvane/`
`bitflip-mutation.svg`	„Bit-flip mutation“	`/operatori/mutaciya/`
`swap-inversion.svg`	„Swap и inversion mutation за пермутации“	`/operatori/mutaciya/`, `/predstavyane/permutacionni/`
`convergence-plot-template.png`	„Best/mean fitness по поколения“	`/parametri/sxodimost-diversitet/`
`diversity-heterozygosity-plot.png`	„Разнообразие: хетерозиготност по поколения“	`/parametri/sxodimost-diversitet/`
`pareto-front-illustration.png`	„Pareto front и недоминирани решения“	`/razshireniya/multiobjective/`
`nsga2-flow.svg`	„NSGA-II: mating + survival selection“	`/razshireniya/multiobjective/nsga-ii/`
`bbob-24-functions-table.png`	„BBOB: 24 функции (таблично)“	`/ocenyavane/benchmarks/coco-bbob/`
`ecdf-runtime.svg`	„ECDF на runtime (ERT targets)“	`/ocenyavane/metriki/ednocel/`
`wfg-transformations.svg`	„WFG toolkit: bias, multimodality, non-separability“	`/ocenyavane/benchmarks/zdt-dtlz-wfg/`

Тези активи са съгласувани с ключовите „концептуални визуализации“ в класическата литература (процес, оператори, diversity/convergence) и с практиката на benchmarking рамки като COCO/BBOB. [163]

1 2 12 15 38 https://mitpress.mit.edu/9780262581110/adaptation-in-natural-and-artificial-systems/
https://mitpress.mit.edu/9780262581110/adaptation-in-natural-and-artificial-systems/

3 4 7 8 9 13 24 25 28 29 53 55 58 59 70 71 73 74 124 131 134 138 150 159 160 163 https://www.ibug.doc.ic.ac.uk/media/uploads/documents/courses/GeneticAlgorithm-tutorial.pdf
https://www.ibug.doc.ic.ac.uk/media/uploads/documents/courses/GeneticAlgorithm-tutorial.pdf

5 6 10 60 https://dl.acm.org/doi/10.5555/534133
https://dl.acm.org/doi/10.5555/534133

11 14 16 61 141 146 155 https://p0p0v.com/science/downloads/Popov03.pdf
https://p0p0v.com/science/downloads/Popov03.pdf

17 18 31 63 92 101 115 125 127 130 135 153 https://sci2s.ugr.es/sites/default/files/files/TematicWebSites/EAMHCO/contributionsGECCO09/bbobdocexperiment.pdf
https://sci2s.ugr.es/sites/default/files/files/TematicWebSites/EAMHCO/contributionsGECCO09/bbobdocexperiment.pdf

19 20 21 22 34 35 https://www.cs.ubc.ca/~hutter/earg/papers07/00585893.pdf
https://www.cs.ubc.ca/~hutter/earg/papers07/00585893.pdf

23 77 118 119 120 129 https://www.uni-sofia.bg/index.php/bul/content/download/89255/688277/version/1/file/Vellev_autoref_final.pdf
https://www.uni-sofia.bg/index.php/bul/content/download/89255/688277/version/1/file/Vellev_autoref_final.pdf

26 37 47 48 126 148 https://pubsonline.informs.org/doi/10.1287/ijoc.6.2.154
https://pubsonline.informs.org/doi/10.1287/ijoc.6.2.154

27 30 132 152 https://direct.mit.edu/books/monograph/5437/The-Simple-Genetic-AlgorithmFoundations-and-Theory
https://direct.mit.edu/books/monograph/5437/The-Simple-Genetic-AlgorithmFoundations-and-Theory

32 33 https://www.researchgate.net/publication/3642792_Convergence_of_evolutionary_algorithms_in_general_search_spaces
https://www.researchgate.net/publication/3642792_Convergence_of_evolutionary_algorithms_in_general_search_spaces

36 93 102 140 https://numbbo.github.io/coco/testsuites/bbob
https://numbbo.github.io/coco/testsuites/bbob

39 103 https://pypi.org/project/deap/1.2.1/
https://pypi.org/project/deap/1.2.1/

40 https://mitpress.mit.edu/9780262631853/an-introduction-to-genetic-algorithms/
https://mitpress.mit.edu/9780262631853/an-introduction-to-genetic-algorithms/

41 42 44 56 57 117 142 157 https://www.complex-systems.com/abstracts/v09_i02_a02/
https://www.complex-systems.com/abstracts/v09_i02_a02/

43 116 https://e-university.tu-sofia.bg/e-publ/files/1064_Yordanova_Georgieva_article_E_Eotchet.pdf
https://e-university.tu-sofia.bg/e-publ/files/1064_Yordanova_Georgieva_article_E_Eotchet.pdf

45 https://eldorado.tu-dortmund.de/bitstreams/51f73e97-24cb-4d7f-928e-30c73968046b/download
https://eldorado.tu-dortmund.de/bitstreams/51f73e97-24cb-4d7f-928e-30c73968046b/download

46 128 https://www.egr.msu.edu/~kdeb/papers/k2012016.pdf
https://www.egr.msu.edu/~kdeb/papers/k2012016.pdf

49 50 51 https://mitpress.mit.edu/9780262527910/genetic-programming/
https://mitpress.mit.edu/9780262527910/genetic-programming/

52 https://deap.readthedocs.io/en/master/tutorials/advanced/gp.html
https://deap.readthedocs.io/en/master/tutorials/advanced/gp.html

54 67 68 144 https://cs.adelaide.edu.au/~zbyszek/Papers/self9.pdf
https://cs.adelaide.edu.au/~zbyszek/Papers/self9.pdf

62 https://www.p0p0v.com/science/downloads/Popov03a.pdf
https://www.p0p0v.com/science/downloads/Popov03a.pdf

64 66 78 79 80 83 136 162 https://sci2s.ugr.es/sites/default/files/files/Teaching/OtherPostGraduateCourses/Metaheuristicas/Deb_NSGAII.pdf
https://sci2s.ugr.es/sites/default/files/files/Teaching/OtherPostGraduateCourses/Metaheuristicas/Deb_NSGAII.pdf

65 104 https://tik-old.ee.ethz.ch/file//3ec604450bf683daaf27f9027e69f44d/DTLZ2004a.pdf
https://tik-old.ee.ethz.ch/file//3ec604450bf683daaf27f9027e69f44d/DTLZ2004a.pdf

69 https://www.cs.vu.nl/~gusz/papers/2007-eib-mich-schoen-smit-chap.pdf
https://www.cs.vu.nl/~gusz/papers/2007-eib-mich-schoen-smit-chap.pdf

72 85 86 110 147 https://esa.int/gsp/ACT/open_source/pagmo/
https://esa.int/gsp/ACT/open_source/pagmo/

75 https://esa.github.io/pygmo/
https://esa.github.io/pygmo/

76 https://esa.github.io/pygmo2/
https://esa.github.io/pygmo2/

81 87 94 https://sop.tik.ee.ethz.ch/publicationListFiles/ztlf2003a.pdf
https://sop.tik.ee.ethz.ch/publicationListFiles/ztlf2003a.pdf

82 https://www.egr.msu.edu/~kdeb/papers/k2011003.pdf
https://www.egr.msu.edu/~kdeb/papers/k2011003.pdf

84 https://pymoo.org/algorithms/moo/nsga2.html
https://pymoo.org/algorithms/moo/nsga2.html

88 90 https://eckartzitzler.ch/img/publications/abbz2009a.pdf
https://eckartzitzler.ch/img/publications/abbz2009a.pdf

89 91 https://github.com/Project-Platypus/Platypus
https://github.com/Project-Platypus/Platypus

95 96 98 139 https://sop.tik.ee.ethz.ch/publicationListFiles/zdt2000a.pdf
https://sop.tik.ee.ethz.ch/publicationListFiles/zdt2000a.pdf

97 100 https://www.researchgate.net/publication/3418888_A_review_of_multiobjective_test_problems_and_a_scalable_test_problem_toolkit
https://www.researchgate.net/publication/3418888_A_review_of_multiobjective_test_problems_and_a_scalable_test_problem_toolkit

99 https://www.research-collection.ethz.ch/bitstreams/4d08c69c-9f97-424d-bbf6-7a647bcb3d13/download
https://www.research-collection.ethz.ch/bitstreams/4d08c69c-9f97-424d-bbf6-7a647bcb3d13/download

105 106 107 108 143 https://deap.readthedocs.io/
https://deap.readthedocs.io/

109 145 161 https://pygad.readthedocs.io/
https://pygad.readthedocs.io/

111 https://www.egr.msu.edu/~kdeb/papers/c2020001.pdf
https://www.egr.msu.edu/~kdeb/papers/c2020001.pdf

112 133 137 https://jmetal.readthedocs.io/
https://jmetal.readthedocs.io/

113 154 https://moeaframework.org/
https://moeaframework.org/

114 https://jenetics.io/manual/manual-7.2.0.pdf
https://jenetics.io/manual/manual-7.2.0.pdf

121 122 123 https://www.bfu.bg/uploads/MyDocuments/summaries_pg.pdf
https://www.bfu.bg/uploads/MyDocuments/summaries_pg.pdf

149 https://pymoo.org/
https://pymoo.org/

151 https://platypus.readthedocs.io/
https://platypus.readthedocs.io/

156 https://github.com/GMUEClab/ecj
https://github.com/GMUEClab/ecj

158 https://jenetics.io/
https://jenetics.io/